

COLLEGE ADMISSION → TOR SYSTEM

Frontend Development Guide

UI/UX blueprint aligned with the secure backend, complete grading engine, GPA/GWA, academic lifecycle, graduation and Transcript of Records.

Frontend principle: make workflows clear and efficient while the backend remains authoritative for permissions, validation, calculations and official records.

Companion document to the Backend, Security & Complete Grading Development Guide.

1. Frontend Architecture

Recommended structure

Layer	Responsibility
App shell	Routing, auth state, navigation, layout and responsive structure.
Pages	Role-specific workflows and information views.
Components	Reusable forms, tables, cards, dialogs, tabs, status badges and uploads.
Data layer	API calls, caching, loading/error states and synchronization.
Design system	Consistent typography, spacing, controls, tables and accessibility.
Security	Frontend never replaces backend authorization.

Suggested stack: React/Next.js or equivalent, TypeScript, reusable component system, typed API contracts, schema-based forms, responsive CSS and automated component/integration/E2E testing.

2. Global Application Shell

Required global UI

Area	Behavior
Header	Institution identity, user, notifications, help and account menu.
Navigation	Role-aware menu; backend still enforces permissions.
Breadcrumbs	Context for complex registrar/academic workflows.
Page header	Title, context, status, filters and actions.
Content	Responsive cards, tables, forms and workflow panels.
Search	Only search data the user is authorized to search.
Feedback	Success, warning, error, loading and empty states.

3. Authentication & Accounts

Screens

- Login
- Password reset and recovery
- Account verification
- MFA setup/challenge
- Session-expired
- Unauthorized/forbidden
- Account/session security

Privileged accounts should use MFA according to backend policy. Never ship database credentials or service-role keys to the browser.

4. Admissions / Applicant Portal

Core screens

- Public landing page
- Program catalog
- Admissions requirements/deadlines

- Application form
- Applicant dashboard
- Requirement checklist
- Document upload/re-upload
- Exam/interview status
- Admission decision
- Acceptance/confirmation

The applicant should always see a clear status and next action rather than needing to understand internal workflow states.

5. Student Portal

Navigation/modules

Module	Main screens
Dashboard	Term, program, enrollment, standing, alerts and deadlines.
Profile	Personal/contact information and permitted updates.
Enrollment	Term, course/section selection, schedule and status.
Courses	Current subjects, instructors, rooms and schedules.
Grades	Assessment results, final grades, term/cumulative GPA/GWA.
Progress	Degree audit and remaining requirements.
Documents	Records, certificates and credential access.
Finance	Assessment/payment/holds where integrated.
Scholarship/OJT/Research	Status, requirements, milestones and submissions.
Graduation	Application, clearance, requirements and status.
Notifications	Institution messages and workflow updates.

6. Enrollment UX

Guided workflow

Step	UI
1	Select term and review enrollment period.
2	Review curriculum/recommended courses.
3	Search/select courses and sections.
4	Display backend prerequisite, schedule, capacity, hold and policy validation.
5	Review timetable/calendar.
6	Review assessment information.
7	Confirm selections.
8	Submit enrollment transaction.
9	Show pending/approved/rejected/official result.

Add/drop/withdrawal should use dedicated dialogs, deadlines, consequence summaries and confirmation. The frontend never decides whether the request is allowed.

7. COMPLETE GRADING FRONTEND

The grading UI must expose the backend grading engine without making the browser authoritative for official calculations.

Instructor gradebook

UI	Function
Section header	Course, section, term and instructor.
Student roster	Searchable/filterable enrolled students.
Assessment columns	Configured quizzes, exams, projects, laboratories, etc.
Score cells	Keyboard-friendly entry with validation.
Special states	Distinguish zero, missing, excused and other configured states.
Computed result	Backend-calculated running/final grade.
Actions	Save draft, calculate, submit, review permitted history.

Gradebook UX

- Sticky student-name column
- Horizontal scrolling for wide gradebooks
- Paste/import with validation preview
- Clear saved/unsaved/invalid states
- Bulk actions only when authorized
- Confirmation before final submission
- Official grades become read-only except through correction workflow

Grade calculation preview

Assessment → score → percentage → weight → contribution → raw/computed percentage → transmutation → rounding → final grade → grade state

- Show grading scheme/version where appropriate
- Warn about missing required components
- Distinguish provisional from official values
- Request authoritative recalculation from backend

Grade workflow states

State	Frontend
Draft	Editable by authorized instructor.
Submitted	Show submitted; editing restricted.
Under review	Show review status/feedback.
Approved	Show approval state.
Posted/Official	Read-only official result.
Correction required	Authorized correction workflow.
Corrected	Current official result plus permitted history indicator.

Student grades

- Term selector
- Course/units/final grade/status
- Assessment breakdown when released
- Term GPA/GWA

- Cumulative GPA/GWA
- Academic standing
- Repeat indicators where appropriate
- Clear official/provisional distinction

8. GPA/GWA, Standing & Degree Audit

GPA/GWA screens

- Term-by-term GPA/GWA
- Cumulative GPA/GWA
- Units attempted/earned where applicable
- Calculation period
- Academic standing
- Standing history where permitted

Official GPA/GWA must come from backend results; the browser may display and explain them but should not be the authority.

Degree audit

Area	Display
Program/curriculum	Program and curriculum version.
Progress	Completion and remaining requirements.
General education	Satisfied/in-progress/missing.
Major requirements	Satisfied/in-progress/missing.
Electives	Completed/remaining.
Units	Required/completed/in-progress/remaining.
Prerequisites	Warnings/dependencies.
Substitutions/waivers	Approved exceptions.
Graduation readiness	Backend-derived result.

9. Registrar / Academic Staff

Dashboards

- Enrollment activity
- Pending admissions/enrollment/grade requests
- Grade submission/approval queues
- Graduation applications
- TOR/credential requests
- Academic record corrections
- Document verification
- Operational alerts

Admissions staff

- Application queue
- Applicant profile

- Requirement checklist
- Document review
- Verification
- Exam/interview status
- Admission decision
- Communication history
- Audit/history

Academic admin/adviser

- Programs/curricula
- Courses/prerequisites
- Academic calendar
- Sections/schedules
- Advising
- Degree audit
- Substitutions/waivers
- Standing/progress
- Authorized rule configuration

10. Graduation, TOR & Credentials

Screens

Screen	Function
Graduation application	Submit and track application.
Eligibility	Display backend degree-audit result.
Clearance	Track required clearances.
Requirements	Checklist with document/status.
Review	Authorized staff review/approval.
Conferment	Degree conferment status.
Credentials	Diploma/TOR/certification request and issuance.
Verification	Secure credential verification page.

11. Finance, Scholarship, OJT & Research

Role-specific interfaces

- Assessment/payment/hold status
- Scholarship status and requirements
- OJT/practicum placement and milestones
- Research/thesis/capstone proposals and approvals
- Document submissions
- Reviewer comments
- Completion/clearance status

12. Notifications & Communication

Notification center

- Read/unread state
- Deadline reminders
- Grade-release notifications
- Enrollment alerts
- Admission decisions
- Approval/rejection messages
- Graduation/TOR updates
- Email/SMS/push through backend-controlled integrations

13. Forms, Tables & Search

UX requirements

- Clear labels/helper text
- Field-level validation
- Server-side pagination
- Authorized server-side filtering/search
- Loading/empty/error states
- Safe exports only for authorized users
- Do not load all student records into the browser merely to filter them

14. Design System & Components

Group	Components
Navigation	Sidebar, top bar, breadcrumbs, tabs.
Actions	Buttons, icon buttons, menus.
Forms	Inputs, selects, autocomplete, checkbox/radio, date/time, file upload.
Feedback	Alerts, toasts, dialogs, progress.
Data	Tables, pagination, filters, cards, statistics, timelines.
Academic	Grade cells, grade badges, GPA cards, degree-progress indicators.
Workflow	Status badges, steppers, approval panels, history timelines.
Documents	Upload, list, preview/download.
Accessibility	Focus states, labels, keyboard support, semantic structure.

15. Responsive Design & Accessibility

Requirements

- Mobile-first student/applicant workflows
- Tablet/desktop layouts for staff
- Responsive tables or card alternatives
- Large-screen gradebook optimized for data entry
- Touch and keyboard-friendly forms

- Semantic HTML
- Keyboard navigation
- Visible focus
- Do not rely on color alone
- Accessible validation/errors
- Readable contrast/typography
- Screen-reader-friendly tables/forms
- Support zoom without breaking essential actions

16. Frontend Security

Rules

- Never expose service-role keys/database passwords/private signing secrets.
- Do not use hidden UI controls as authorization.
- Use secure backend authentication/session mechanisms.
- Treat API responses as untrusted data.
- Render user-generated content safely.
- Do not unnecessarily store sensitive academic data in browser storage.
- Clear sensitive cached state on logout/session expiry.
- Handle unauthorized and expired sessions consistently.

17. Performance & State

Performance

- Lazy-load large staff modules
- Paginate large datasets
- Cache stable reference data only when safe
- Debounce search
- Virtualize very large gradebooks/tables where necessary
- Optimize document previews
- Monitor frontend performance

State model

- Server state: students, enrollment, grades, sections, applications, degree audit and credentials.
- Session state: identity and session status.
- Local UI state: dialogs, filters, tabs, drafts and temporary selections.

After mutations such as enrollment, grade posting or credential issuance, invalidate/refresh affected server state.

18. Key Routes

```
/login
/applicant/dashboard
/student/dashboard
/student/enrollment
/student/grades
/student/progress
```

```

/student/documents
/student/graduation
/instructor/gradebook
/registrar/dashboard
/registrar/students
/registrar/grades
/registrar/graduation
/registrar/credentials
/admin/programs
/admin/curricula
/admin/courses
/admin/sections
/verify/{credential}

```

19. Testing Strategy

Test	Examples
Component	Forms, grade cells, tables, dialogs, status badges.
Integration	Login → dashboard → enrollment → confirmation.
Grading E2E	Scores → calculate → submit → approve → post → student sees official grade.
Authorization	Unauthorized routes/actions do not expose sensitive data.
Degree audit	Correct satisfied/missing/substituted display.
TOR	Request → processing → issuance → verification.
Responsive	Mobile, tablet and desktop.
Accessibility	Keyboard, screen reader, focus, contrast and errors.
Performance	Large gradebooks, student lists, searches and dashboards.

20. Development Roadmap

Phase	Frontend deliverables
1. Foundation	Design system, app shell, auth, routing, API client, error/loading patterns.
2. Public + Admissions	Website, program catalog, application, requirements, applicant dashboard.
3. Student Core	Dashboard, profile, enrollment, schedule, documents, notifications.
4. Academic	Courses, grades, GPA/GWA, standing, degree audit.
5. Staff	Registrar, admissions, academic admin, adviser and instructor dashboards.
6. Complete Grading	Gradebook, assessments, calculation preview, submission, approval, correction.
7. Graduation + TOR	Graduation, clearance, credentials and verification.
8. Production UX	Accessibility, responsive hardening, performance and security.

21. Production Readiness Checklist

- Every backend workflow has a usable frontend flow.
- Role-aware navigation is clear while backend authorization remains authoritative.
- Enrollment displays backend validation results.
- Gradebook supports assessment entry, calculation preview, submission and correction.
- GPA/GWA and degree audit use authoritative backend results.
- Official grades are visually distinct from drafts/provisional values.
- TOR/credential workflows are complete.
- Large datasets support search/filter/pagination.
- Forms have validation, error recovery and unsaved-change handling.
- Mobile/tablet/desktop layouts are tested.
- Accessibility testing is complete.

- Sensitive data is not unnecessarily persisted in browser storage.
- No service/database secrets are shipped to the client.
- Session expiry and unauthorized access are handled safely.
- Automated tests cover critical workflows.

22. Core Frontend Principles

- The frontend is a workflow interface, not the source of truth.
- Backend permissions and business rules remain authoritative.
- Complex academic processes should be presented as clear guided steps.
- Official grades and credentials must be visually distinguishable from drafts.
- High-impact actions should explain their consequences before confirmation.
- Reusable components should prevent duplicate screen logic.
- Accessibility, responsiveness and performance are production requirements.